

Experiment 8

1 Python Code

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

L = 4
T_sym = 1
N_sym = 8
beta = 0.5
SNR_dB_range = np.array([0, 2, 4, 6, 8, 10, 14, 16])

nSamples = 2 * L
nTraces = 100

img = cv2.imread('cameraman.png', cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))

bits = np.unpackbits(img.flatten())
print(f"Total bits formed: {len(bits)}")

a_i = 2 * bits.astype(float) - 1

plt.figure(figsize=(3,3))
plt.scatter(a_i[:100], np.zeros(100))
plt.title("BPSK Constellation")
plt.show()

u_n = np.zeros(len(a_i) * L)
u_n[:L] = a_i

t = np.arange(-N_sym/2, N_sym/2 + 1/L, 1/L)

def get_srrc(t, beta, Ts):

    p = np.zeros(len(t))
```

```

for i in range(len(t)):
    ti = t[i]

    if np.isclose(ti, 0.0):
        p[i] = (1/np.sqrt(Ts))*(1 + beta*(4/np.pi - 1))

    elif beta!=0 and np.isclose(abs(ti), Ts/(4*beta)):
        p[i] = (beta/np.sqrt(2*Ts))*(
            (1 + 2/np.pi)*np.sin(np.pi/(4*beta)) +
            (1 - 2/np.pi)*np.cos(np.pi/(4*beta))
        )
    else:
        num = (np.sin(np.pi*ti*(1-beta)/Ts) +
            (4*beta*ti/Ts)*np.cos(np.pi*ti*(1+beta)/Ts))

        den = (np.pi*ti/Ts)*(1-(4*beta*ti/Ts)**2)

        p[i] = (1/np.sqrt(Ts))*(num/den)

return p

p_n = get_srrc(t, beta, T_sym)

s_n = np.convolve(u_n, p_n, mode='full')

N_filter = len(p_n)
delta = (N_filter - 1) // 2
total_delay = 2 * delta

avg_pwr_s = np.mean(s_n**2)

ber_list = []
reconstructed_images = []

for snr_db in SNR_dB_range:

    snr_lin = 10**(snr_db/10)

    p_noise = avg_pwr_s / snr_lin

    noise = np.sqrt(p_noise)*np.random.randn(len(s_n))

    r_n = s_n + noise

    y_n = np.convolve(r_n, p_n, mode='full')

    a_hat = y_n[total_delay : total_delay + len(a_i)*L : L]

```

```

bits_hat = (a_hat[:len(bits)] >= 0).astype(int)

err = np.sum(bits != bits_hat)

ber_list.append(err/len(bits))

rec_img = np.packbits(bits_hat).reshape(256,256)

reconstructed_images.append(rec_img)

def plot_eye(signal, L, nSamples, nTraces, title):

    plt.figure(figsize=(5,4))

    for i in range(nTraces):

        start = i*L

        segment = signal[start:start+nSamples]

        if len(segment)==nSamples:
            plt.plot(segment, 'b', alpha=0.3)

    plt.title(title)
    plt.grid(True)

snr_eye = [0, 4, 8]

for snr_db in snr_eye:

    snr_lin = 10**((snr_db/10))
    p_noise = avg_pwr_s/snr_lin

    noise = np.sqrt(p_noise)*np.random.randn(len(s_n))

    r_n = s_n + noise

    y_n = np.convolve(r_n, p_n, mode='full')

    plot_eye(y_n, L, nSamples, nTraces,
             f"Eye_Diagram(beta=0.4, SNR={snr_db} dB)")

beta_list = [0.1,0.5,0.9]
snr_fixed = 4

```

```

for b in beta_list:

    p_beta = get_srrc(t, b, T_sym)

    s_beta = np.convolve(u_n,p_beta,'full')

    snr_lin = 10**(snr_fixed/10)

    noise = np.sqrt(np.mean(s_beta**2)/snr_lin)*np.random.randn(len(
        s_beta))

    r_beta = s_beta + noise

    y_beta = np.convolve(r_beta,p_beta,'full')

    plot_eye(y_beta,L,nSamples,nTraces,
             f"Eye_Diagram_(beta={b}_ ,_SNR=4_dB)")

plt.tight_layout()
plt.show()
plt.figure(figsize=(16,12))

plt.subplot(3,4,1)
plt.plot(t,p_n,'g')
plt.title("SRRC_Pulse_Shape_( =0.4)")
plt.grid(True)

plt.subplot(3,4,2)
plt.semilogy(SNR_dB_range,ber_list,'r-o')
plt.xlim(0,8)
plt.title("BER_vs_SNR")
plt.grid(True,which='both')

plt.subplot(3,4,3)
plt.imshow(img,cmap='gray')
plt.title("Original_Image")
plt.axis('off')

for i in range(len(SNR_dB_range)):

    plt.subplot(3,4,i+4)

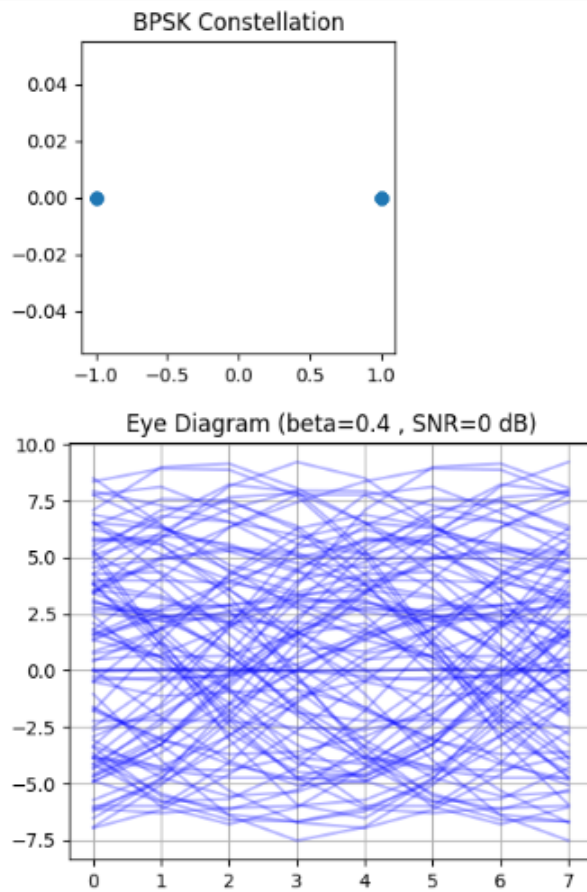
    plt.imshow(reconstructed_images[i],cmap='gray')

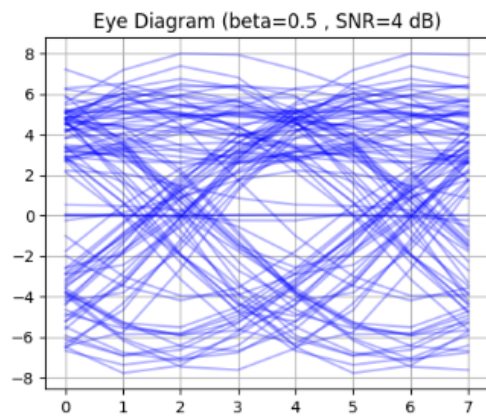
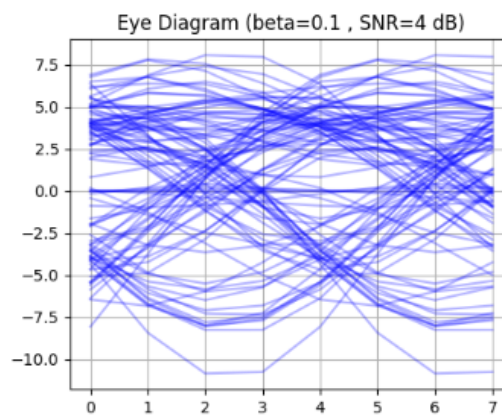
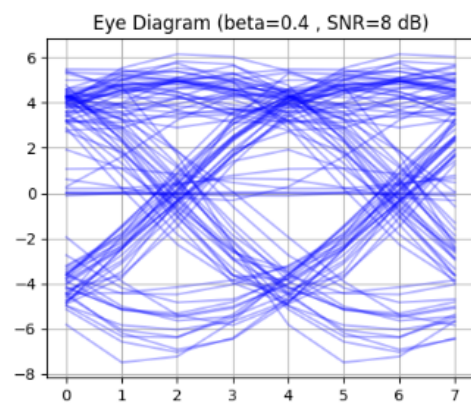
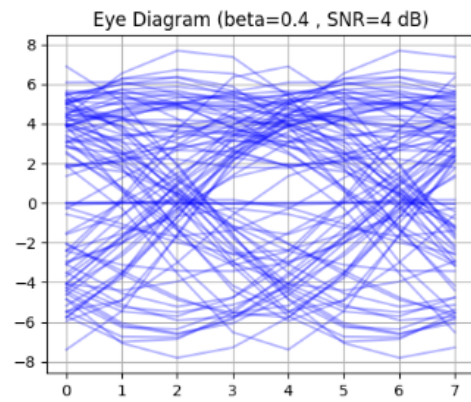
    plt.title(f"SNR={SNR_dB_range[i]}_dB")

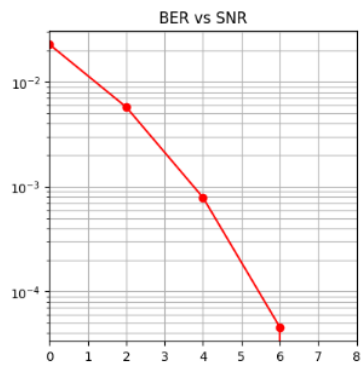
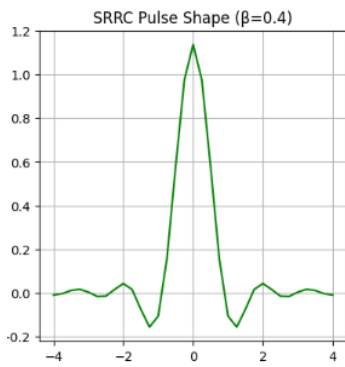
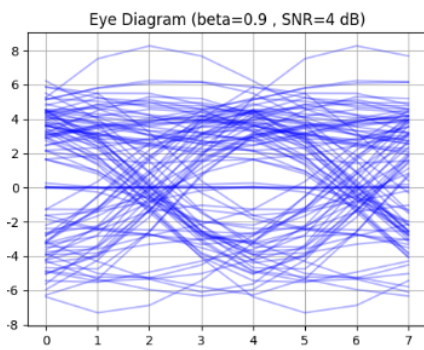
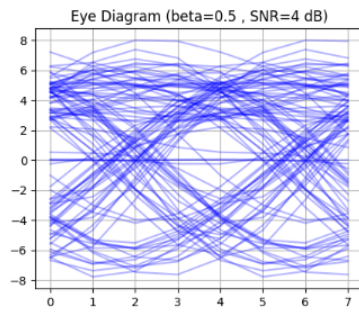
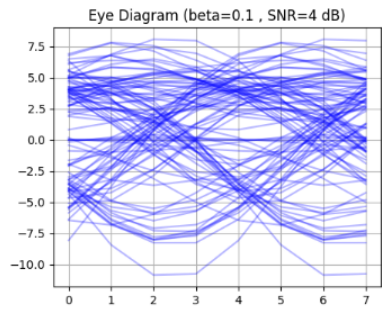
```

```
plt.axis('off')  
  
plt.tight_layout()  
plt.show()
```

2 Reconstructed Images







SNR=2 dB



SNR=4 dB



SNR=6 dB



SNR=8 dB



SNR=10 dB



SNR=14 dB



SNR=16 dB

